Contenedores con Docker: guía práctica en 9 ejemplos

Ejemplos sueltos, cada uno con sus comandos, lo que debes ver y lo que has aprendido, para entender qué es un contenedor antes del laboratorio de S2 (API del curso en Docker). Pensada para hacerla **tú solo en tu Codespace**, a tu ritmo.

> **Estado de esta guía.** Los comandos y las salidas son reales: se ejecutaron con Docker 29.4.2 sobre Ubuntu 26.04.1 LTS (GitHub Actions, 10/11/2026). **Todavía no se han ejecutado en un Codespace del curso.** En tu Codespace la versión de Docker puede ser otra: las columnas de `docker image ls`, los tiempos y los tamaños pueden diferir. Los identificadores largos (`…`) y las marcas de tiempo (`"ts": …`) se han acortado porque cambian en cada ejecución. Si algo no sale como aquí, copia el mensaje exacto: es lo que hay que comparar.

Cómo usar esta guía

- **Dónde:** en la terminal de tu Codespace (si aún no lo tienes abierto, empieza por la guía de Codespaces).
- **Cuánto: obligatorio antes de S2**, los ejemplos 1 a 6 (≈60 min): el laboratorio de S2 da por sabido lo que enseñan. Recomendado, la guía completa (los 9, ≈97 min) y el laboratorio de roturas. Son estimaciones, sin cronometrar con alumnado.
- **Cómo:** copia cada comando tal cual, compara con la salida de la guía y lee el «qué ha pasado». Al final de cada ejemplo hay **qué has aprendido** y una pregunta para pensar.
- **Dos presentaciones acompañan esta guía:** Contenedores: el modelo mental (ejemplos 1, 2, 3 y 7) y Del Dockerfile a producción (ejemplos 4, 5, 6, 8 y 9).
- **Después:** el laboratorio «romperlo a propósito» (≈34 min), para practicar el diagnóstico.

Los nueve ejemplos de un vistazo

Nº	Ejemplo	Min	Carpeta	Qué te llevas
1	Imagen y contenedor	8	(ninguna)	Qué es una imagen, qué es un contenedor y por qué un contenedor «desaparece» cuando termina.
2	Ciclo de vida y lo que se pierde	10	(ninguna)	Arrancar, mirar los registros, entrar, parar y volver a arrancar; y comprobar que lo escrito dentro desaparece con el contenedor.
3	Puertos: llegar al contenedor desde fuera	10	(ninguna)	Por qué sin `-p` no se llega al servicio, qué hace `-p 8000:8000` y por qué la API escucha en `0.0.0.0`.
4	Tu primera imagen: Dockerfile y capas	10	`04-primera-imagen`	Construir una imagen de cuatro líneas y ver que está hecha de capas.
5	La caché de capas: construir sin esperar	12	`05-cache`	Dos Dockerfiles que hacen lo mismo con distinto orden. Uno reinstala las dependencias a cada cambio; el otro no.

Nº	Ejemplo	Min	Carpeta	Qué te llevas
6	Configuración y secretos: variables de entorno	10	`06-entorno`	Pasar configuración al contenedor sin reconstruir la imagen, y por qué un secreto nunca va dentro de ella.
7	Volúmenes y montajes: datos que sobreviven	10	(ninguna)	Dos formas de que algo exista fuera del contenedor: un volumen que gestiona Docker y una carpeta tuya montada dentro.
8	Contexto, `.dockerignore`, tamaño y usuario no root	12	`08-contexto`	Qué entra en la imagen sin que lo pienses, cómo evitarlo y por qué la imagen del curso no se ejecuta como root.
9	Una API en un contenedor: salud, registros y límites	15	`09-api-minima`	Una API pequeña (sin librerías) para ver, en miniatura, lo que hace la imagen del curso: comprobación de salud, registros JSON, usuario no root y límite de memoria.

Las carpetas están en `01-Sesiones/S2-API-y-Docker/contenedores/ejemplos/`. Los ejemplos sin carpeta se hacen en una carpeta vacía que crea el primer comando.

Antes de empezar

Comprueba que Docker responde (si falla, no sigas: avisa con el mensaje exacto):

```
docker version
```

Debe mostrar dos bloques, **Client** y **Server**. Si solo aparece el cliente y el servidor da error, Docker no está en marcha en tu Codespace.

Ejemplo 1 · Imagen y contenedor

8 min · Qué es una imagen, qué es un contenedor y por qué un contenedor «desaparece» cuando termina.

Colócate en la carpeta:

```
mkdir -p ~/practica-docker && cd ~/practica-docker
```

1. Descarga la imagen: una plantilla de solo lectura con Linux y Python 3.11. La primera vez tarda; luego ya está en tu máquina.

```
docker pull python:3.11-slim
```

Salida:

```
Digest: sha256:07d4824036af…
Status: Downloaded newer image for python:3.11-slim
docker.io/library/python:3.11-slim
```

(Solo se muestran las últimas 3 líneas.)

2. Lista las imágenes que tienes. Una imagen no se ejecuta: se usa para crear contenedores. (**Disk usage** es lo que ocupa descomprimida en disco; **content size**, lo que se descargó. Según tu versión de Docker las columnas pueden variar.)

```
docker image ls python:3.11-slim
```

Salida:

```
IMAGE              ID             DISK USAGE   CONTENT SIZE   EXTRA
python:3.11-slim   07d4824036af        189MB         47.8MB
```

3. Crea un contenedor desde la imagen, ejecuta un comando dentro, imprime y se borra solo (`--rm`).

```
docker run --rm python:3.11-slim python -c "import sys; print(sys.version)"
```

Salida:

```
3.11.18 (main, Nov  5 2026, 07:57:53) [GCC 14.2.0]
```

4. Dentro hay otro Linux, aunque tu Codespace sea otra cosa. Lo que ves es la imagen, no tu máquina.

```
docker run --rm python:3.11-slim sh -c "head -2 /etc/os-release"
```

Salida:

```
PRETTY_NAME="Debian GNU/Linux 13 (trixie)"
NAME="Debian GNU/Linux"
```

5. Lista todos los contenedores, también los parados. Está vacía: los dos anteriores se borraron al terminar (`--rm`).

```
docker ps -a
```

Salida:

```
CONTAINER ID   IMAGE     COMMAND   CREATED   STATUS    PORTS     NAMES
```

6. Esta vez sin `--rm` y con nombre. El proceso termina enseguida, pero el contenedor se queda.

```
docker run --name hola python:3.11-slim python -c "print('hola desde un contenedor')"
```

Salida:

```
hola desde un contenedor
```

7. Sigue ahí, parado: `Exited (0)` significa que su proceso terminó bien.

```
docker ps -a --format "table {{.Names}}\t{{.Image}}\t{{.Status}}"
```

Salida:

```
NAMES     IMAGE              STATUS
hola      python:3.11-slim   Exited (0) Less than a second ago
```

8. Borra el contenedor (la imagen sigue intacta).

```
docker rm hola
```

Salida:

```
hola
```

Qué has aprendido

- Una **imagen** es la plantilla de solo lectura; un **contenedor** es una ejecución de esa imagen. De una imagen salen tantos contenedores como quieras.
- Un contenedor **vive lo que vive su proceso**: cuando el comando termina, el contenedor se para (`Exited`).
- `--rm` borra el contenedor al terminar; sin él se acumulan y hay que borrarlos con `docker rm`.

Para pensar

- ¿Qué pasa con la imagen cuando borras un contenedor?
- ¿Por qué `docker ps` (sin `-a`) no mostraba `hola` aunque existiera?

Ejemplo 2 · Ciclo de vida y lo que se pierde

10 min · Arrancar, mirar los registros, entrar, parar y volver a arrancar; y comprobar que lo escrito dentro desaparece con el contenedor.

Colócate en la carpeta:

```
mkdir -p ~/practica-docker && cd ~/practica-docker
```

1. `-d` lo deja en segundo plano y devuelve su identificador. El proceso (un bucle) no termina nunca.

```
docker run -d --name reloj python:3.11-slim sh -c "while true; do echo tic; sleep 1; done"
```

Salida:

```
bb6179882561…
```

2. Lo que el proceso escribe en pantalla (stdout) queda guardado: es donde la API deja sus registros.

Espera unos 3 segundos desde el comando anterior.

```
docker logs --tail 3 reloj
```

Salida:

```
tic
tic
tic
```

3. `exec` ejecuta otro comando **dentro** del contenedor que ya está en marcha.

```
docker exec reloj sh -c "ls / | head -5"
```

Salida:

```
bin
boot
dev
etc
home
```

4. Pide al proceso que pare y, si no lo hace, lo mata a los 10 s. Aquí tarda **10 segundos**: el bucle de `sh` no atiende la señal de parada, y Docker lo mata (verás `Exited (137)`). Una API como uvicorn sí la atiende y para en un instante.

```
docker stop reloj
```

Salida:

```
reloj
```

5. Parado, no borrado. `137` = 128 + 9: el proceso terminó por la señal 9 (KILL).

```
docker ps -a --format "table {{.Names}}\t{{.Status}}"
```

Salida:

```
NAMES     STATUS
reloj     Exited (137) Less than a second ago
```

6. Un contenedor parado se puede volver a arrancar: conserva su configuración.

```
docker start reloj
```

Salida:

```
reloj
```

7. Vuelve a estar en marcha.

```
docker ps --format "table {{.Names}}\t{{.Status}}"
```

Salida:

```
NAMES     STATUS
reloj     Up 1 second
```

8. `-f` lo para y lo borra de una vez.

```
docker rm -f reloj
```

Salida:

```
reloj
```

9. Escribe un archivo dentro del contenedor y lo lee: funciona.

```
docker run --name efimero python:3.11-slim sh -c "echo dato > /tmp/nota.txt && cat /tmp/nota.txt"
```

Salida:

```
dato
```

10. Borramos ese contenedor.

```
docker rm efimero
```

Salida:

```
efimero
```

11. Un contenedor nuevo, de la misma imagen, no tiene el archivo: lo escrito dentro murió con el contenedor anterior.

```
docker run --rm python:3.11-slim cat /tmp/nota.txt
```

Este comando falla a propósito: termina con un código distinto de 0.

Salida:

```
cat: /tmp/nota.txt: No such file or directory
```

Qué has aprendido

- `docker logs` enseña lo que el proceso escribe en la salida estándar; por eso la API registra en JSON por pantalla, no en un archivo.
- `exec` es tu forma de «entrar» a mirar; `stop` / `start` conservan el contenedor; `rm` lo destruye.
- **Lo que se escribe dentro de un contenedor desaparece con él.** Para que algo sobreviva hay que usar un volumen (ejemplo 7) o guardarlo fuera.

Para pensar

- ¿Dónde guardarías las predicciones que genera una API si el contenedor puede borrarse en cualquier momento?
- ¿Qué diferencia hay entre `docker stop` y `docker rm`?

Ejemplo 3 · Puertos: llegar al contenedor desde fuera

10 min · Por qué sin `-p` no se llega al servicio, qué hace `-p 8000:8000` y por qué la API escucha en `0.0.0.0`.

Colócate en la carpeta:

```
mkdir -p ~/practica-docker && cd ~/practica-docker
```

1. Un servidor web mínimo que escucha en el puerto 8000 **dentro** del contenedor. No publicamos ningún puerto.

```
docker run -d --name web python:3.11-slim python -m http.server 8000
```

Salida:

```
9aae11d11a31…
```

2. El servicio funciona, pero desde tu máquina no hay camino hasta él.

Espera unos 2 segundos desde el comando anterior.

```
curl -sS --max-time 3 http://127.0.0.1:8000/
```

Este comando falla a propósito: termina con un código distinto de 0.

Salida:

```
curl: (7) Failed to connect to 127.0.0.1 port 8000 after 0 ms: Could not connect to server
```

3. Lo borramos para repetirlo publicando el puerto.

```
docker rm -f web
```

Salida:

```
web
```

4. `-p 8000:8000` conecta el puerto 8000 de tu máquina (izquierda) con el 8000 del contenedor (derecha).

```
docker run -d --name web -p 8000:8000 python:3.11-slim python -m http.server 8000
```

Salida:

```
ee777137c6ff…
```

5. Ahora responde 200.

Espera unos 2 segundos desde el comando anterior.

```
curl -sS -o /dev/null -w "%{http_code}\n" http://127.0.0.1:8000/
```

Salida:

```
200
```

6. Muestra qué puertos están publicados.

```
docker port web
```

Salida:

```
8000/tcp -> 0.0.0.0:8000
8000/tcp -> [::]:8000
```

7. Borramos el anterior.

```
docker rm -f web
```

Salida:

```
web
```

8. El mismo servidor, pero escuchando **solo en 127.0.0.1 dentro del contenedor**. Publicamos el puerto igual.

```
docker run -d --name web -p 8000:8000 python:3.11-slim python -m http.server 8000 --bind 127.0.0.1
```

Salida:

```
6f9954c7a1a7…
```

9. El puerto está publicado y aun así falla: el tráfico entra por la red del contenedor, no por su «127.0.0.1», y el servicio no lo atiende.

Espera unos 2 segundos desde el comando anterior.

```
curl -sS --max-time 3 http://127.0.0.1:8000/
```

Este comando falla a propósito: termina con un código distinto de 0.

Salida:

```
curl: (56) Recv failure: Connection reset by peer
```

10. Limpiamos.

```
docker rm -f web
```

Salida:

```
web
```

Qué has aprendido

- Los puertos de un contenedor no son accesibles hasta que los **publicas** con `-p máquina:contenedor`.

- Dentro del contenedor el servicio debe escuchar en `0.0.0.0` (todas las interfaces); si escucha solo en `127.0.0.1`, nadie de fuera lo alcanza. Por eso la API del curso arranca con `--host 0.0.0.0`.
- Dos contenedores no pueden publicar el mismo puerto de tu máquina (lo verás en la rotura 1).

Para pensar

- La respuesta a la pregunta del laboratorio de S2: ¿por qué el contenedor de la API escucha en `0.0.0.0` y no en `127.0.0.1`?

Ejemplo 4 · Tu primera imagen: Dockerfile y capas

10 min · Construir una imagen de cuatro líneas y ver que está hecha de capas.

Colócate en la carpeta:

```
cd "$(git rev-parse --show-toplevel)/01-Sesiones/S2-API-y-Docker/contenedores/ejemplos/04-primera-imagen"
```

Archivos de la carpeta (léelos antes de ejecutar nada):

`Dockerfile`

```
FROM python:3.11-slim
WORKDIR /app
COPY app.py .
CMD ["python", "app.py"]
```

`app.py`

```
import platform

print("Hola desde un contenedor con Python", platform.python_version())
```

1. Construye la imagen `saludo:1` a partir del `Dockerfile` de esta carpeta (`.` es el **contexto**: la carpeta cuyos archivos puede copiar). Filtramos la salida para ver los **pasos**: uno por instrucción. (En tu terminal verás la salida con barras de progreso; aquí va en texto.)

```
docker build --progress=plain -t saludo:1 . 2>&1 | grep -E "\[[0-9]+/[0-9]+\]|naming to"
```

Salida:

```
#5 [1/3] FROM docker.io/library/python:3.11-slim@sha256:07d4824036af…
#6 [2/3] WORKDIR /app
#7 [3/3] COPY app.py .
#8 naming to docker.io/library/saludo:1 done
```

2. Un contenedor de tu imagen: ejecuta `app.py`.

```
docker run --rm saludo:1
```

Salida:

```
Hola desde un contenedor con Python 3.11.18
```

3. Las capas, de la última a la primera: `CMD`, `COPY`, `WORKDIR` son las tuyas; debajo está la imagen base.

```
docker history --format "{{.Size}}\t{{.CreatedBy}}" saludo:1 | head -5
```

Salida:

```
0B  CMD ["python" "app.py"]
12.3kB  COPY app.py .
8.19kB  WORKDIR /app
0B  CMD ["python3"]
16.4kB  RUN /bin/sh -c set -eux;  for src in idle3 p…
```

4. Tamaño de la imagen base…

```
docker image ls python:3.11-slim
```

Salida:

```
IMAGE              ID             DISK USAGE   CONTENT SIZE   EXTRA
python:3.11-slim   07d4824036af        189MB         47.8MB
```

5. …y el de la tuya: casi lo mismo. Casi todo el peso viene de la base; tus cuatro líneas añaden muy poco.

```
docker image ls saludo:1
```

Salida:

```
IMAGE      ID             DISK USAGE   CONTENT SIZE   EXTRA
saludo:1   a16f43127cc8        187MB         45.6MB
```

Qué has aprendido

- Un **Dockerfile** es una receta: cada instrucción (`FROM`, `WORKDIR`, `COPY`, `RUN`…) añade una **capa** a la imagen.
- `FROM` elige la base; casi todo el tamaño viene de ella. `CMD` es lo que se ejecuta al crear un contenedor.
- El **contexto** (`.`) es lo que Docker puede copiar a la imagen: solo lo que esté en esa carpeta.

Para pensar

- ¿Qué cambiarías para que imprima otro saludo? ¿Basta con editar `app.py`, o hay que volver a construir?

Ejemplo 5 · La caché de capas: construir sin esperar

12 min · Dos Dockerfiles que hacen lo mismo con distinto orden. Uno reinstala las dependencias a cada cambio; el otro no.

Colócate en la carpeta:

```
cd "$(git rev-parse --show-toplevel)/01-Sesiones/S2-API-y-Docker/contenedores/ejemplos/05-cache"
```

Archivos de la carpeta (léelos antes de ejecutar nada):

`Dockerfile.bueno`

```
# Orden BUENO: primero lo que casi nunca cambia (las dependencias), al final lo que cambia siempre (el codigo).
FROM python:3.11-slim
WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY app.py .
CMD ["python", "app.py"]
```

`Dockerfile.malo`

```
# Orden MALO: el codigo se copia antes de instalar las dependencias.
# Cualquier cambio en app.py invalida la capa del COPY y, con ella, la del pip install.
FROM python:3.11-slim
WORKDIR /app
COPY . .
RUN pip install --no-cache-dir -r requirements.txt
CMD ["python", "app.py"]
```

`app.py`

```
import numpy as np

print("numpy", np.__version__, "->", int(np.arange(10).sum()))
```

`requirements.txt`

```
numpy==1.26.4
```

1. `Dockerfile.malo` copia todo el código y **después** instala las dependencias. Primera construcción: instala todo (el `WORKDIR` puede salir como `CACHED` porque el ejemplo anterior ya creó esa capa).

```
docker build --progress=plain -f Dockerfile.malo -t cache:malo . 2>&1 | grep -E "\[[0-9]+/[0-9]+\]|CACHED"
```

Salida:

```
#4 [1/4] FROM docker.io/library/python:3.11-slim@sha256:07d4824036af…
#5 [2/4] WORKDIR /app
#5 CACHED
#7 [3/4] COPY . .
#8 [4/4] RUN pip install --no-cache-dir -r requirements.txt
```

2. Cambiamos una sola línea del código.

```
echo "# cambio" >> app.py
```

3. Se vuelve a ejecutar `RUN pip install`: el `COPY . .` cambió y arrastra todas las capas siguientes. Cada cambio de código cuesta una reinstalación completa.

```
docker build --progress=plain -f Dockerfile.malo -t cache:malo . 2>&1 | grep -E "\[[0-9]+/[0-9]+\]|CACHED"
```

Salida:

```
#4 [1/4] FROM docker.io/library/python:3.11-slim@sha256:07d4824036af…
#6 [2/4] WORKDIR /app
#6 CACHED
#7 [3/4] COPY . .
#8 [4/4] RUN pip install --no-cache-dir -r requirements.txt
```

4. `Dockerfile.bueno` copia **solo** `requirements.txt`, instala, y copia el código al final. Primera construcción: instala.

```
docker build --progress=plain -f Dockerfile.bueno -t cache:bueno . 2>&1 | grep -E "\[[0-9]+/[0-9]+\]|CACHED"
```

Salida:

```
#5 [1/5] FROM docker.io/library/python:3.11-slim@sha256:07d4824036af…
#6 [2/5] WORKDIR /app
#6 CACHED
#7 [3/5] COPY requirements.txt .
#8 [4/5] RUN pip install --no-cache-dir -r requirements.txt
#9 [5/5] COPY app.py .
```

5. Otro cambio de código.

```
echo "# otro cambio" >> app.py
```

6. Ahora `RUN pip install` sale con **CACHED**: Docker reutiliza la capa porque `requirements.txt` no cambió. Solo se rehace el `COPY app.py`.

```
docker build --progress=plain -f Dockerfile.bueno -t cache:bueno . 2>&1 | grep -E "\[[0-9]+/[0-9]+\]|CACHED"
```

Salida:

```
#5 [1/5] FROM docker.io/library/python:3.11-slim@sha256:07d4824036af…
#6 [3/5] COPY requirements.txt .
#6 CACHED
#7 [2/5] WORKDIR /app
#7 CACHED
#8 [4/5] RUN pip install --no-cache-dir -r requirements.txt
#8 CACHED
#9 [5/5] COPY app.py .
```

7. La imagen funciona igual.

```
docker run --rm cache:bueno
```

Salida:

```
numpy 1.26.4 -> 45
```

8. Quita las líneas que añadimos para dejar `app.py` como estaba.

```
sed -i '/^# /d' app.py
```

Qué has aprendido

- Docker reutiliza una capa si **ella y todas las anteriores** no han cambiado. Un cambio invalida esa capa y todas las siguientes.
- **Regla de oro:** primero lo que casi nunca cambia (dependencias), al final lo que cambia siempre (tu código). Es lo que hace el `Dockerfile` del proyecto: `COPY requirements-api.lock` + `pip install`, y luego `COPY ia_mlops/`.
- En un equipo, esto se traduce en minutos ahorrados en cada construcción, local y en CI.

Para pensar

- En el `Dockerfile` del proyecto, ¿qué capa se reconstruye si cambias una línea de `ia_mlops/`? ¿Y si cambias `requirements-api.lock`?

Ejemplo 6 · Configuración y secretos: variables de entorno

10 min · Pasar configuración al contenedor sin reconstruir la imagen, y por qué un secreto nunca va dentro de ella.

Colócate en la carpeta:

```
cd "$(git rev-parse --show-toplevel)/01-Sesiones/S2-API-y-Docker/contenedores/ejemplos/06-entorno"
```

Archivos de la carpeta (léelos antes de ejecutar nada):

`Dockerfile`

```
FROM python:3.11-slim
WORKDIR /app
COPY app.py .
ENV SALUDO="hola por defecto"
CMD ["python", "app.py"]
```

`Dockerfile.mal`

```
# MAL: el valor queda grabado en la imagen. Cualquiera que reciba la imagen puede leerlo.
# (El valor es inventado: nunca pongas uno real.)
FROM python:3.11-slim
WORKDIR /app
COPY app.py .
ENV TOKEN_API="no-es-un-secreto-real-123"
CMD ["python", "app.py"]
```

`app.py`

```
import os

print("saludo:", os.environ.get("SALUDO", "(sin definir)"))
print("token presente:", "TOKEN_API" in os.environ)
```

1. Una imagen que lee dos variables de entorno. (`-q` solo imprime el identificador de la imagen.)

```
docker build -q -t entorno:1 .
```

Salida:

```
sha256:efe1f66e628a…
```

2. Sin pasar nada: usa el valor por defecto que lleva la imagen y no ve el token.

```
docker run --rm entorno:1
```

Salida:

```
saludo: hola por defecto
token presente: False
```

3. `-e` pasa una variable **al arrancar**, sin tocar la imagen.

```
docker run --rm -e SALUDO="hola desde -e" -e TOKEN_API=abc entorno:1
```

Salida:

```
saludo: hola desde -e
token presente: True
```

4. Un archivo con las variables (`mi.env` está en `.gitignore`: no se sube).

```
printf 'SALUDO=desde archivo\nTOKEN_API=xyz\n' > mi.env
```

5. `--env-file` pasa todas las del archivo.

```
docker run --rm --env-file mi.env entorno:1
```

Salida:

```
saludo: desde archivo
token presente: True
```

6. Ahora la forma **incorrecta**: `ENV TOKEN_API=...` dentro del Dockerfile. La construcción funciona, pero Docker ya te avisa de que es mala idea.

```
docker build -f Dockerfile.mal -t entorno:mal . 2>&1 | grep SecretsUsedInArgOrEnv
```

Salida:

```
#1 WARN: SecretsUsedInArgOrEnv: Do not use ARG or ENV instructions for sensitive data (ENV
"TOKEN_API") (line 6)
 - SecretsUsedInArgOrEnv: Do not use ARG or ENV instructions for sensitive data (ENV "TOKEN_API")
(line 6)
```

7. El historial de la imagen lo enseña en claro…

```
docker history --no-trunc --format "{{.CreatedBy}}" entorno:mal | grep TOKEN_API
```

Salida:

```
ENV TOKEN_API=no-es-un-secreto-real-123
```

8. ...y `inspect` también. Quien tenga la imagen tiene el «secreto».

```
docker inspect --format "{{.Config.Env}}" entorno:mal | grep -o "TOKEN_API=[^ ]*"
```

Salida:

```
TOKEN_API=no-es-un-secreto-real-123
```

9. Borra el archivo de ejemplo.

```
rm -f mi.env
```

Qué has aprendido

- La **configuración** (qué modelo, qué puerto, qué entorno) se pasa al arrancar con `-e` o `--env-file`: la misma imagen sirve para desarrollo y producción.
- Un **secreto** (clave, contraseña, token) **nunca** va en `ENV`, en un `COPY` ni en la imagen: queda grabado en sus capas y se puede leer.
- En la nube se inyectan desde un gestor de secretos (en GCP, Secret Manager, que veremos en S3) y llegan como variables o archivos montados al arrancar.

Para pensar

- El `.dockerignore` del proyecto excluye `**/.env`, `**/*.key` y `**/credenciales*.json`. ¿Qué evita eso exactamente?

Ejemplo 7 · Volúmenes y montajes: datos que sobreviven

10 min · Dos formas de que algo exista fuera del contenedor: un volumen que gestiona Docker y una carpeta tuya montada dentro.

Colócate en la carpeta:

```
mkdir -p ~/practica-docker && cd ~/practica-docker
```

1. Un volumen es un almacenamiento que Docker guarda aparte de cualquier contenedor.

```
docker volume create datos
```

Salida:

```
datos
```

2. `-v datos:/datos` monta el volumen en `/datos`. Escribimos un archivo y el contenedor se borra.

```
docker run --rm -v datos:/datos python:3.11-slim sh -c "echo version 1 > /datos/nota.txt"
```

3. Otro contenedor, distinto, ve el mismo archivo: el dato sobrevivió.

```
docker run --rm -v datos:/datos python:3.11-slim cat /datos/nota.txt
```

Salida:

```
version 1
```

4. Ahora una carpeta **tuya** con un «modelo».

```
mkdir -p modelo-demo && echo "modelo v1" > modelo-demo/modelo.txt
```

5. Montamos esa carpeta dentro del contenedor, en **solo lectura** (`:ro`).

```
docker run --rm -v "$PWD/modelo-demo:/modelos:ro" python:3.11-slim cat /modelos/modelo.txt
```

Salida:

```
modelo v1
```

6. Cambiamos el archivo en tu máquina…

```
echo "modelo v2" > modelo-demo/modelo.txt
```

7. …y el contenedor ve el cambio **sin reconstruir ninguna imagen**.

```
docker run --rm -v "$PWD/modelo-demo:/modelos:ro" python:3.11-slim cat /modelos/modelo.txt
```

Salida:

```
modelo v2
```

8. Con `:ro` el contenedor no puede escribir en tu carpeta.

```
docker run --rm -v "$PWD/modelo-demo:/modelos:ro" python:3.11-slim sh -c "echo x >
/modelos/otro.txt"
```

Este comando falla a propósito: termina con un código distinto de 0.

Salida:

```
sh: 1: cannot create /modelos/otro.txt: Read-only file system
```

9. Limpieza.

```
docker volume rm datos && rm -rf modelo-demo
```

Salida:

```
datos
```

Qué has aprendido

- **Volumen** (`-v nombre:ruta`): lo gestiona Docker; sirve para datos que deben sobrevivir al contenedor (una base de datos, resultados).
- **Montaje de carpeta** (`-v carpeta:ruta`): expone una carpeta tuya dentro; sirve para desarrollo y para cambiar datos o modelo sin reconstruir. `:ro` evita que el contenedor la modifique.
- El proyecto del curso **copia el modelo dentro de la imagen** (`COPY models/...`): la imagen es autosuficiente y reproducible. Montarlo sería más flexible pero depende de que la carpeta esté donde toca.

Para pensar

- ¿Qué ventaja y qué riesgo tiene montar el modelo en lugar de copiarlo en la imagen?

Ejemplo 8 · Contexto, `.dockerignore`, tamaño y usuario no root

12 min · Qué entra en la imagen sin que lo pienses, cómo evitarlo y por qué la imagen del curso no se ejecuta como root.

Colócate en la carpeta:

```
cd "$(git rev-parse --show-toplevel)/01-Sesiones/S2-API-y-Docker/contenedores/ejemplos/08-contexto"
```

Archivos de la carpeta (léelos antes de ejecutar nada):

`Dockerfile`

```
FROM python:3.11-slim
WORKDIR /app
COPY . .
CMD ["python", "app.py"]
```

`Dockerfile.noroot`

```
FROM python:3.11-slim
WORKDIR /app
COPY app.py .
RUN useradd --uid 10001 --no-create-home --shell /usr/sbin/nologin appuser
USER 10001
CMD ["python", "app.py"]
```

`app.py`

```
import os

print("el proceso corre con el usuario numero", os.getuid())
```

1. Creamos un archivo de 50 MB que **no** tiene nada que ver con la aplicación.

```
dd if=/dev/zero of=datos-grandes.bin bs=1M count=50 status=none && du -h datos-grandes.bin
```

Salida:

```
50M datos-grandes.bin
```

2. Docker envía **toda la carpeta** (el contexto) al motor de construcción, incluido el archivo grande (la línea con MB), y el `COPY . .` lo mete en la imagen.

```
docker build --progress=plain -t contexto:con-basura . 2>&1 | grep -E "transferring context|COPY
\. \."
```

Salida:

```
#3 transferring context: 2B done
#5 transferring context: 52.44MB 0.1s done
#7 [3/3] COPY . .
```

3. La imagen engordó por un archivo que no usa: mira **DISK USAGE** (en nuestra ejecución, 240 MB frente a 187 MB de la imagen limpia que verás después). **CONTENT SIZE** casi no cambia solo porque un archivo lleno de ceros se comprime hasta casi nada.

```
docker image ls contexto:con-basura
```

Salida:

```
IMAGE                ID             DISK USAGE   CONTENT SIZE   EXTRA
contexto:con-basura  111b7a2c64de        240MB         45.7MB
```

4. El `.dockerignore` lista lo que Docker debe **ignorar**.

```
printf 'datos-grandes.bin\n' > .dockerignore
```

5. Mismo Dockerfile: el contexto ya no incluye el archivo grande (solo unos cientos de bytes).

```
docker build --progress=plain -t contexto:limpio . 2>&1 | grep -E "transferring context|COPY \.
\."
```

Salida:

```
#3 transferring context: 58B done
#4 transferring context: 178B done
#7 [3/3] COPY . .
```

6. Y la imagen vuelve a su tamaño.

```
docker image ls contexto:limpio
```

Salida:

```
IMAGE             ID             DISK USAGE   CONTENT SIZE   EXTRA
contexto:limpio   551966da0d9e        187MB         45.6MB
```

7. Por defecto el proceso corre como **root** (usuario 0) dentro del contenedor.

```
docker run --rm contexto:limpio
```

Salida:

```
el proceso corre con el usuario numero 0
```

8. `Dockerfile.noroot` crea un usuario y cambia a él con `USER`. (`-q` solo imprime el identificador de la imagen.)

```
docker build -q -f Dockerfile.noroot -t contexto:noroot .
```

Salida:

```
sha256:a671aec5abe1…
```

9. Ahora corre con el usuario 10001, sin privilegios: si alguien explota la aplicación, tiene menos margen.

```
docker run --rm contexto:noroot
```

Salida:

```
el proceso corre con el usuario numero 10001
```

10. Limpieza.

```
rm -f datos-grandes.bin .dockerignore
```

Qué has aprendido

- `COPY . .` copia **todo lo que haya en el contexto**. Datos, claves, `.git`, cuadernos… todo acaba en la imagen salvo que lo excluyas en `.dockerignore`.
- Un `.dockerignore` bien hecho hace la imagen más pequeña, la construcción más rápida y **evita filtrar secretos**. El del proyecto excluye `data/`, `notebooks/`, `tests/`, `.git`, claves…
- Por defecto un contenedor corre como **root**. Producción exige un usuario sin privilegios (`USER`): es lo que hace la imagen del curso (usuario 10001).

Para pensar

- Mira el `.dockerignore` de `04-Proyecto-Base/`. ¿Qué carpeta dejaría de ir en la imagen si la quitaras de ahí? ¿Y qué pasaría si `models/` estuviera en la lista?

Ejemplo 9 · Una API en un contenedor: salud, registros y límites

15 min · Una API pequeña (sin librerías) para ver, en miniatura, lo que hace la imagen del curso: comprobación de salud, registros JSON, usuario no root y límite de memoria.

Colócate en la carpeta:

```
cd "$(git rev-parse --show-toplevel)/01-Sesiones/S2-API-y-Docker/contenedores/ejemplos/09-api-minima"
```

Archivos de la carpeta (léelos antes de ejecutar nada):

`Dockerfile`

```
FROM python:3.11-slim
ENV PYTHONUNBUFFERED=1
WORKDIR /app
COPY api.py .
RUN useradd --uid 10001 --no-create-home --shell /usr/sbin/nologin apiuser
USER 10001
EXPOSE 8080
HEALTHCHECK --interval=3s --timeout=2s --start-period=1s --retries=2 \
    CMD python -c "import urllib.request; urllib.request.urlopen('http://127.0.0.1:8080/salud', timeout=1)"
CMD ["python", "api.py"]
```

`api.py`

```
"""API minima con la biblioteca estandar: lo justo para ver healthcheck, logs y limites de un contenedor."""
import json
import os
import time
from http.server import BaseHTTPRequestHandler, HTTPServer

MODELO = os.environ.get("MODELO", "v1")


def log(**campos):
    print(json.dumps({"ts": round(time.time(), 3), **campos}, ensure_ascii=False), flush=True)


class Manejador(BaseHTTPRequestHandler):
    def _responder(self, codigo, cuerpo):
        datos = json.dumps(cuerpo).encode()
        self.send_response(codigo)
        self.send_header("Content-Type", "application/json")
        self.send_header("Content-Length", str(len(datos)))
        self.end_headers()
        self.wfile.write(datos)
        log(metodo=self.command, ruta=self.path, codigo=codigo)

    def do_GET(self):
        if self.path == "/salud":
            self._responder(200, {"estado": "vivo"})
        elif self.path == "/modelo":
            self._responder(200, {"modelo": MODELO})
        else:
            self._responder(404, {"detalle": "no existe"})

    def log_message(self, *args):  # el registro de acceso por defecto no es JSON: se usa log()
        pass


if __name__ == "__main__":
    puerto = int(os.environ.get("PUERTO", "8080"))
    log(evento="arranque", puerto=puerto, modelo=MODELO)
    HTTPServer(("0.0.0.0", puerto), Manejador).serve_forever()
```

1. Imagen de una API de 40 líneas. Mira el `Dockerfile`: tiene `USER`, `EXPOSE` y un `HEALTHCHECK`.

```
docker build -q -t api-minima:1 .
```

Salida:

```
sha256:d72852dee7da…
```

2. Publica el puerto 8081, pasa la configuración (`MODELO`) y **limita la memoria a 128 MB**.

```
docker run -d --name mini -p 8081:8080 -e MODELO=v2 --memory 128m api-minima:1
```

Salida:

```
6de2857694af…
```

3. Recién arrancado: Docker aún no sabe si está sano (`health: starting`).

```
docker ps --format "table {{.Names}}\t{{.Status}}\t{{.Ports}}"
```

Salida:

```
NAMES     STATUS                                       PORTS
mini      Up Less than a second (health: starting)   0.0.0.0:8081->8080/tcp, [::]:8081->8080/tcp
```

4. Unos segundos después: el `HEALTHCHECK` del Dockerfile ha comprobado `/salud` y marca `healthy`. Es lo que usan los orquestadores para saber si reiniciarlo.

Espera unos 6 segundos desde el comando anterior.

```
docker ps --format "table {{.Names}}\t{{.Status}}\t{{.Ports}}"
```

Salida:

```
NAMES     STATUS                    PORTS
mini      Up 6 seconds (healthy)   0.0.0.0:8081->8080/tcp, [::]:8081->8080/tcp
```

5. La configuración llegó por la variable de entorno.

```
curl -s http://127.0.0.1:8081/modelo
```

Salida:

```
{"modelo": "v2"}
```

6. Una ruta que no existe responde 404.

```
curl -s -o /dev/null -w "%{http_code}\n" http://127.0.0.1:8081/nada
```

Salida:

```
404
```

7. Los registros son JSON por pantalla, una línea por petición (incluidas las del healthcheck): fáciles de recoger y de filtrar.

```
docker logs mini
```

Salida:

```
{"ts": …, "evento": "arranque", "puerto": 8080, "modelo": "v2"}
{"ts": …, "metodo": "GET", "ruta": "/salud", "codigo": 200}
{"ts": …, "metodo": "GET", "ruta": "/modelo", "codigo": 200}
{"ts": …, "metodo": "GET", "ruta": "/nada", "codigo": 404}
```

8. Memoria que usa frente al límite que le pusimos.

```
docker stats --no-stream --format "table {{.Name}}\t{{.MemUsage}}\t{{.MemPerc}}" mini
```

Salida:

```
NAME      MEM USAGE / LIMIT   MEM %
mini      13.97MiB / 128MiB   10.91%
```

9. El proceso no es root.

```
docker exec mini id -u
```

Salida:

```
10001
```

10. Limpieza.

```
docker rm -f mini
```

Salida:

```
mini
```

Qué has aprendido

- Una imagen de producción es mucho más que «que arranque»: **usuario sin privilegios**, `HEALTHCHECK`, **configuración por variables**, **registros por stdout** y **límites de recursos**. La imagen de la API del curso hace todo esto.
- `docker ps` muestra la salud; `docker logs` los registros; `docker stats` el consumo; `docker exec` te deja mirar dentro.
- Un límite de memoria que se supera mata el contenedor (lo verás en la rotura 4): un modelo grande necesita memoria suficiente.

Para pensar

- Compara este `Dockerfile` con `04-Proyecto-Base/deploy/docker/Dockerfile`: ¿qué tienen igual y qué añade el del proyecto?

Chuleta de comandos (formas generales)

Para…	Forma general
Descargar una imagen	`docker pull imagen:etiqueta`
Ver imágenes	`docker image ls`
Crear y ejecutar un contenedor	`docker run [opciones] imagen [comando]`
En segundo plano / con nombre / borrar al salir	`-d` / `--name nombre` / `--rm`
Publicar un puerto	`-p puerto-de-tu-máquina:puerto-del-contenedor`
Pasar variables	`-e NOMBRE=valor` o `--env-file archivo`
Montar volumen o carpeta	`-v nombre:/ruta` o `-v carpeta:/ruta[:ro]`
Limitar memoria	`--memory 128m`
Ver contenedores (también parados)	`docker ps` (`-a` para todos)
Ver los registros	`docker logs [--tail n] nombre`
Ejecutar algo dentro	`docker exec nombre comando`
Parar / arrancar / borrar	`docker stop` / `docker start` / `docker rm [-f]`
Construir una imagen	`docker build [-f Dockerfile.x] -t nombre:etiqueta .`
Ver las capas / la configuración	`docker history imagen` / `docker inspect objeto`
Consumo de recursos	`docker stats --no-stream`

Lo que esta guía no cubre (a propósito)

- **Docker Compose**, varias imágenes a la vez, redes personalizadas y registros de imágenes: no hacen falta para S2 y se verán, en lo que toque, en S3.
- **Cloud Run y Artifact Registry** (la nube de GCP): se ven en S3. En la presentación «Del Dockerfile a producción» hay una diapositiva con lo que Google documenta sobre el contenedor en Cloud Run; es lectura de su documentación, **no probada** en este material.